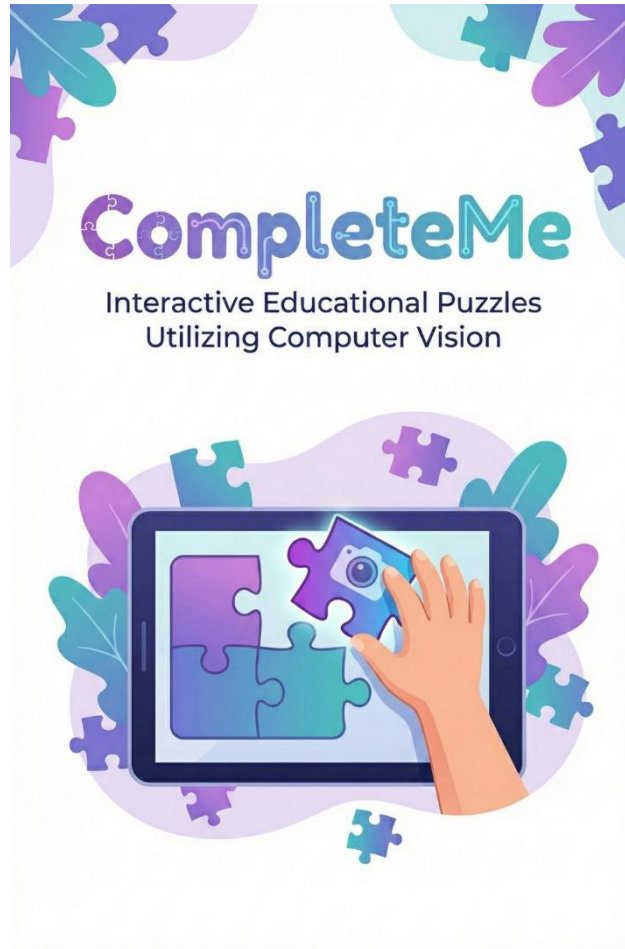


CompleteMe

Interactive Educational Puzzle Platform Utilizing Computer Vision



Final Project Summary

Or Reshef S 324064849 | Gal Azoulay 323859967

Afeka College of Engineering

Final Year Computer Science Project 2025–2026

<https://github.com/orsreshef/CompleteMe>

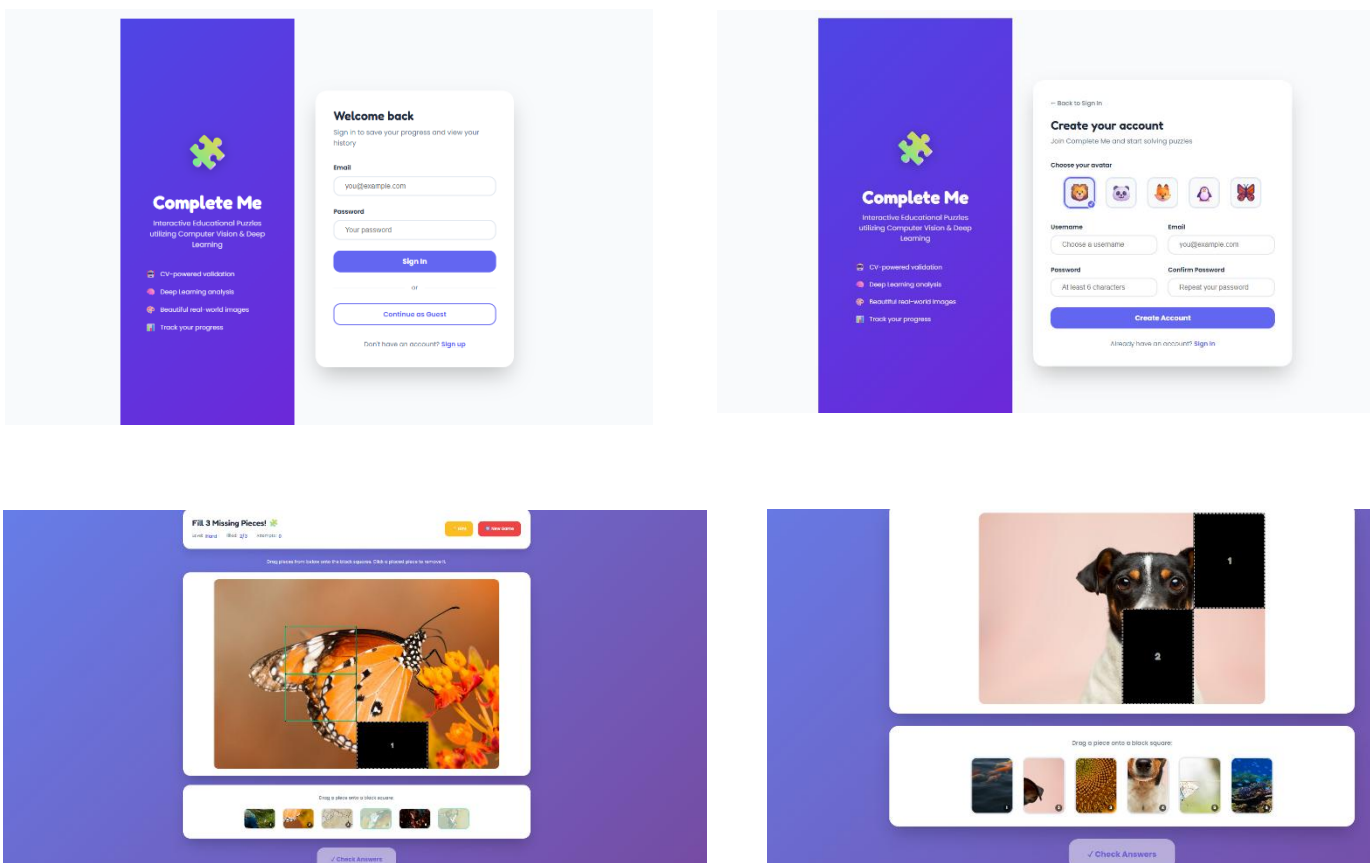
1. Introduction

This document serves as the final summary for our big and final project, CompleteMe - an interactive educational puzzle platform built for children. Its purpose is to provide a clear and complete picture of the finished product: what was built, how it differs from the original plan, what testing was carried out to ensure quality and where all supporting documents can be found in GitHub.

CompleteMe is a web-based puzzle game where users are shown an image with one or more missing regions and must identify the correct piece from a set of options. The core innovation of the project lies in how correctness is determined: rather than relying on simple coordinate or pixel matching, the platform uses a multi-algorithm computer vision pipeline to evaluate boundary colors, texture consistency, and deep visual features extracted through a ResNet50 network. This approach makes the validation academically meaningful and robust.

The platform was designed with children as the primary audience, which shaped decisions around interface simplicity, visual feedback, content safety, and engaging gameplay mechanics such as confetti animations and a points-based scoring system.

More detailed information is available in the separate project documents: the Requirements document, the Detailed Design document, and the Software Test Plan (STP), all of which are linked in Section 5 of this document.



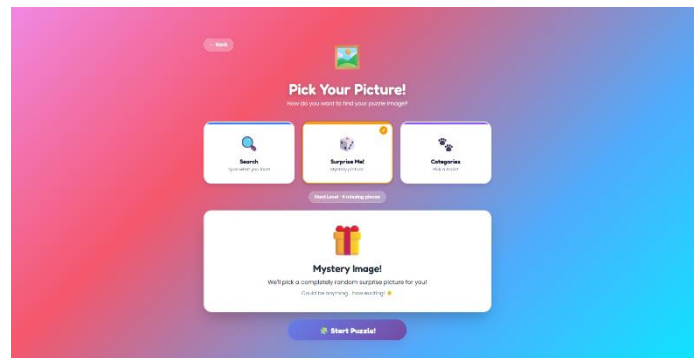
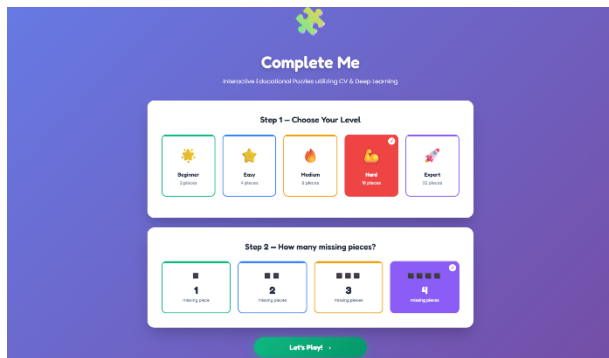
2. Description of the Final Product

The final product is a fully functional web application consisting of a React frontend, a Flask REST API backend and a PostgreSQL database running locally. The computer vision validation pipeline is implemented in Python using PyTorch (ResNet50 via torchvision) and OpenCV and runs as part of the backend server. Images are sourced dynamically from the Unsplash API.

2.1 Core Gameplay Flow

A user - either registered or playing as a guest - arrives at the Welcome screen, where they select a difficulty level (Beginner through Expert, corresponding to grid sizes of 2 through 32 pieces) and the number of missing regions (1 to 4). They then choose an image by typing a keyword, selecting a predefined category, or using the Surprise Me button for a random image. Once the game begins, the backend fetches the image, divides it into a grid, masks the chosen regions with black squares, extracts the correct pieces, generates decoy pieces, and returns everything to the frontend.

On the Game Board, the user sees the masked puzzle and a tray of six-piece options (one correct, the rest decoys). Pieces are placed by drag-and-drop. When all regions are filled, the user clicks Check Answer, and the backend runs the full CV validation pipeline to determine whether each placed piece ranks first among all candidates. Feedback is shown immediately through a validation modal that includes a confidence score with color coding (green for high confidence, red for low), and a confetti animation on success.

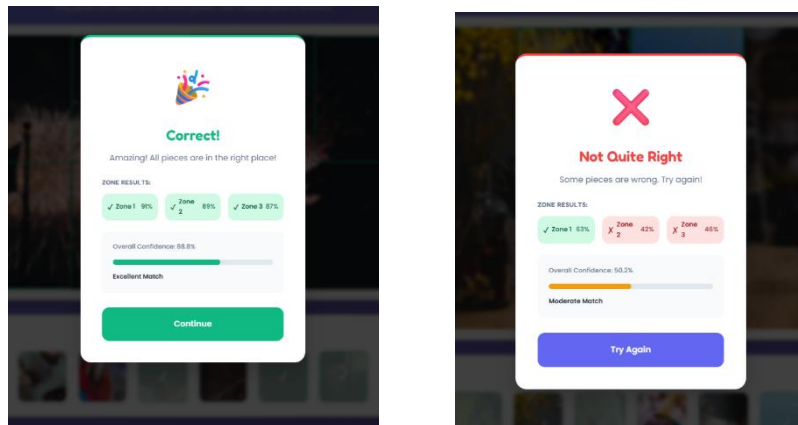


2.2 Computer Vision Validation Pipeline

The validation engine is the academic centerpiece of the project. When a user submits an answer, the backend scores all six candidate pieces using multiple analysis layers and ranks them. A piece is accepted as correct only if it ranks first among all six options. The pipeline consists of the following components:

- **Boundary Matcher:** Extracts thin color strips from the four edges of the black square in the puzzle image and compares them against the corresponding edges of each candidate piece using HSV color histogram analysis. This is the fast primary path.

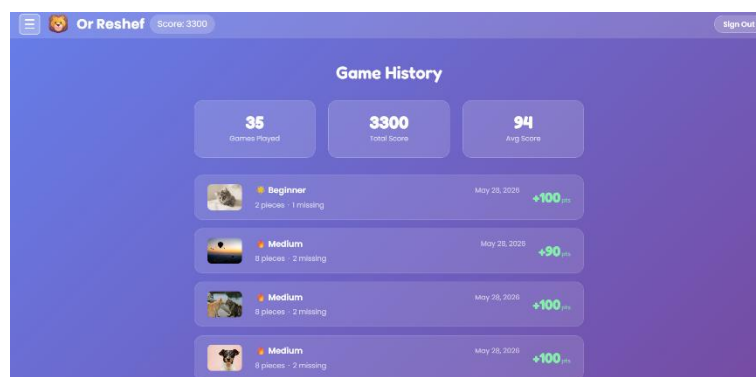
- **Color Analyzer:** Uses K-Means clustering to identify dominant colors in each piece region and compares via histogram matching.
- **Texture Analysis:** Applies Local Binary Patterns (LBP) to evaluate structural pattern consistency across piece boundaries.
- **Deep Feature Extraction:** Loads a pretrained ResNet50 model via torchvision to produce high-dimensional feature vectors. Candidate pieces are compared to the boundary context using cosine similarity.
- **Tie-breaking:** When two candidate scores fall within a margin of 0.02 of each other, a comprehensive validation function is called as a tiebreaker to resolve ambiguous cases.



2.3 User Accounts and Progress Tracking

The platform supports full user authentication: registration with username, email, password, and avatar profile picture. Login with JWT-based sessions using HTTP-only cookies, token refresh and secure logout. Registered users have access to a History screen that displays completed games in reverse chronological order, along with summary statistics (total games, total score, average score). Users can also edit their profile, change their password or delete their account. Guest mode is supported and provides full gameplay without saving progress.

Scores are calculated the way that first-attempt correct answer earns 100 points, the second earns 90, and so on, with a floor of 10 points.



2.4 Additional Features

- **Hint System:** Available when exactly one zone remains unfilled. Clicking the Hint button calls the hint endpoint, which analyzes the boundary colors around the empty slot and returns a human-readable color description to guide the user without revealing the answer.
- **Scoring and points:** Incorrect attempts reduce the score that can be earned, encouraging users to think before placing pieces.
- **Rate limiting:** Login attempts are limited to 5 per 15-minute window. API requests align with the Unsplash Free Tier constraint of 50 requests per hour.
- **Content safety:** All Unsplash API calls use the `content_filter=high` parameter to ensure child-appropriate imagery.
- **Security:** Passwords are hashed using bcrypt, tokens are stored in HTTP-only secure cookies, and all inputs are sanitized to prevent SQL injection and XSS attacks.

3. Changes from the Original Plan

The large majority of what was originally planned was implemented. All Priority 1 (Critical) requirements from the Requirements document were delivered, and most Priority 2 requirements were also completed. The changes described below were deliberate decisions made during development based on time constraints, technical feasibility and improved design choices.

3.1 Deep Learning Layer - Scope Narrowed

The original design document described Layers 4 and 5 of the CV pipeline, deep feature extraction and semantic validation using ResNet50 as optional additions subject to available time. In practice, Layer 4 (ResNet50-based feature extraction) was implemented and integrated into the validation pipeline as one of the scoring components and later reused to power the DL-based hint feature for the final remaining piece. Full semantic validation (Layer 5) was left out of scope, as the combination of boundary matching, color analysis, texture analysis, and deep features already produced robust results without it.

3.2 Session Storage - Simplified

The design document noted that caching original pieces in Flask session or Redis was "not a must" and would be decided later. In the final implementation, piece data is stored in a simple in-process Python dictionary (`active_games`) keyed by game ID, rather than Flask's cookie-based session or an external Redis store. This kept the deployment simple and appropriate for a local deployment project.

3.3 API Endpoint Structure - Refined

The internal API endpoint structure evolved slightly from the initial design. Endpoints were consolidated and renamed to reflect the final routing architecture (e.g., `/api/puzzle/create`, `/api/puzzle/validate`, `/api/puzzle/hint`) rather than the paths listed in the original design document. The functionality remained the same, only the naming and grouping were adjusted for clarity.

3.4 History - Unlimited Rather Than 10-Record Cap

The requirements document specified storing the most recent 10 completed puzzles per user. The final system stores all completed games without a hard cap, displaying them in reverse chronological order on the History screen.

4. Testing and Quality Assurance

Testing was carried out in accordance with the Software Test Plan (STP) produced earlier in the project. The plan outlined four levels of testing - unit, integration, system, and acceptance - covering all major components: the authentication system, game setup and configuration, the game board and drag-and-drop interaction, the CV validation pipeline, and the history and statistics screens.

4.1 Testing Environment

Tests were run on the full local environment: the Flask backend with all Python dependencies, the React frontend served via Node.js and the database populated with test user accounts. All manual testing was performed on Google Chrome, which is the supported browser for this project. Mobile and cross-browser testing was explicitly out of scope, as documented in the STP.

4.2 Testing Approach in Practice

Testing was conducted continuously throughout development: each new feature was verified immediately after implementation, and any issues found were fixed and re-tested before moving on. Towards the end of the project, a full structured walkthrough of all screens and flows was conducted, following the acceptance testing scenarios described in the STP: a first-time user registering and playing their first game, a returning user reviewing their history, and a guest user completing a puzzle without saving progress.

Unit-level verification covered the score calculation formula, password hashing and verification, boundary color strip extraction, and individual CV utility functions. Integration testing verified the full create-validate-record flow across the frontend, backend, and database. The overall outcome was positive: all tested scenarios worked as expected.

4.3 Test Results Summary

The table below summarizes the areas tested and their outcomes.

Area	What Was Tested	Result
User Authentication	Registration, login with correct/incorrect credentials, guest mode access, JWT cookie handling	✓ Pass
Rate Limiting	Account lockout after 5 failed attempts within 15 minutes, correct 429 and wait-time messaging	✓ Pass
Token Refresh & Logout	Access token refresh on expiry, redirect on invalid refresh token, cookie clearing on logout	✓ Pass
Avatar & Profile Editing	Avatar update reflected in the top UserBar immediately.	✓ Pass
Password & Account Deletion	Password change with correct/incorrect current password, account + history deletion on confirmation	✓ Pass
Difficulty & Region Selection	All five difficulty levels mapped to correct piece counts, region selector adjusts to difficulty	✓ Pass
Image Source (3 paths)	Keyword search, category selection, and Surprise Me all returned valid images from Unsplash. Fallback generated locally when API unavailable	✓ Pass
Drag-and-Drop Interaction	Piece placement, zone highlighting on hover, replacement of existing piece, click-to-remove	✓ Pass
Hint System	Available only when one zone is unfilled.	✓ Pass
“Check Answer” Button	Disabled until all zones filled, enabled correctly after last piece placed	✓ Pass
CV Validation for Correct Piece	Correct piece ranked first across all six candidates, confidence score color coding displayed correctly	✓ Pass
CV Validation for Incorrect Piece	Incorrect placement returned a lower rank, feedback shown with appropriate color indicator	✓ Pass
Scoring Formula	Score decreased by 10 per additional attempt, floor of 10 confirmed.	✓ Pass
History Screen	Game displayed in reverse chronological order, summary stats, inaccessible for guests	✓ Pass
Backend API	All endpoints verified.	✓ Pass
Database Constraints	Unique username/email, cascade delete removed all history on account deletion	✓ Pass

4.4 What Was Not Tested

The following areas were explicitly left out of scope in the STP and were not tested: CV model accuracy, cross-browser compatibility, mobile/touch drag-and-drop, performance/load testing, external service internals (the Unsplash API) were assumed to work correctly and were not tested directly.

5. Project Documents - GitHub Links

The following is a full list of all documents submitted over the course of the project, organized by semester. All files are available in our GitHub repository.

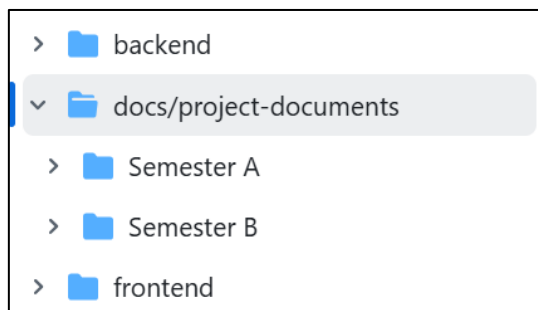
GitHub Repository: <https://github.com/orsreshef/CompleteMe>

Semester A

Submission	Description	File Name
Submission 1	Partner name submission only	-
Submission 2	Project topic description	<i>Assignment 2 – project overview</i>
Submission 3	Literature Review	<i>submission 3 - literature review</i>
Submission 4	Requirements document	<i>submission 4 - requirements</i>
Submission 5	Full Detailed Design document	<i>submission 5 - detailed design</i>
Submission 6	Presentation - in-class lecture	<i>submission 6 - presentation</i>
Submission 7	Peer review document	<i>submission 7 - peer review</i>

Semester B

Submission	Description	File Name
Submission 1	Project progress status	<i>submission1 - prototype</i>
Submission 2	Updated full Detailed Design document	<i>submission 2 - Corrected Detailed Design</i>
Submission 3	Software Test Plan (STP)	<i>submission 3 - STP</i>
Submission 4	Final project poster	<i>submission 4 - poster</i>
Submission 5	Final submission document (this document)	<i>submission 5 - final submission</i>



6. Summary

CompleteMe began as an academic project, and the concept behind it came from a personal motivation: building a puzzle game for our nephews and nieces, one that would let them play with puzzles built from categories they actually love, with a wide variety of images and difficulty options to keep it engaging over time. To meet the academic requirements in a meaningful way, the game validates a piece not by where it was dropped, but by whether its visual content actually fits, using computer vision. Designing a validation engine that is both technically rigorous and practical enough to run in real time during gameplay became the core technical goal of the project.

The final product covers everything that was defined as critical in the original requirements: image retrieval from Unsplash, puzzle generation with multiple difficulty levels and missing region counts, drag-and-drop gameplay, CV-based validation, and ResNet50 deep feature extraction, a hint system and a full user account and history system with authentication.

All tests carried out in line with the Software Test Plan passed successfully. The application is stable, the CV pipeline produces consistent and meaningful results, and the user-facing experience works correctly across all tested scenarios.

